MLX90640 Pi4 connection

11/30/2025

MLX90640 connection to the pi4

Connecting the **MLX90640 thermal camera** to a Raspberry Pi 4 involves both hardware wiring and software setup. Here's a detailed step-by-step guide for your hackathon project:

---

1. Hardware Connection

The MLX90640 communicates via **I²C**, so you'll connect it to the Pi's I²C pins.

MLX90640 Pins:

- **VDD** → 3.3V (Pin 1 on Pi)
- **GND** → Ground (Pin 6 on Pi)
- **SDA** → I²C Data (Pin 3 on Pi, GPIO2)
- **SCL** → I²C Clock (Pin 5 on Pi, GPIO3)
- **AD0** → Connect to GND or leave floating (sets I²C address, default 0x33)
- **INT** → Optional, for interrupts (can be left unconnected initially)

Notes:

- Make sure to use **3.3V**, not 5V.
- Use short wires to reduce I²C noise.
- If multiple MLX90640 modules are used, set different addresses with **AD0**.

---

2. Enable I²C on the Raspberry Pi

1. Open terminal:

```bash
sudo raspi-config
```

2. Go to **Interface Options → I2C → Enable**.
3. Install I²C tools:

```bash
sudo apt update
sudo apt install -y i2c-tools python3-smbus
```

4. Check if sensor is detected:

```bash
```

```
i2cdetect -y 1
```

You should see `0x33` (default MLX90640 address).

---

3. Install Python Libraries

Install the official **Adafruit MLX90640 library**:

bash Copy code

```
pip3 install adafruit-circuitpython-mlx90640
pip3 install numpy matplotlib
```

---

4. Test the Sensor

Here's a basic Python script to read temperature frames:

python Copy code

```python
import time
import board
import busio
import adafruit_mlx90640
import numpy as np
import matplotlib.pyplot as plt

# Setup I2C
i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)
mlx = adafruit_mlx90640.MLX90640(i2c)
mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ  # 2 Hz

frame = np.zeros((24*32,))

plt.ion()
fig, ax = plt.subplots()
im = ax.imshow(frame.reshape((24,32)), cmap='inferno', vmin=20, vmax=40)
plt.colorbar(im)

while True:
    try:
        mlx.getFrame(frame)
        im.set_data(frame.reshape((24,32)))
        plt.draw()
        plt.pause(0.01)
    except KeyboardInterrupt:
        break
```

Explanation:

- The sensor outputs a **24x32 thermal matrix**.
- `RefreshRate` can be set from 0.5Hz to 16Hz.
- You can plot a **heatmap** using Matplotlib for your dashboard.

---

5. Integration with Your Hackathon Setup

Based on your 5-day plan 5_day_hackathon_plan:

- MLX90640 output can be fused with **YOLO human detection** and **RCWL-0516 radar**.
- Generate **heatmap overlays** on camera feeds.
- Send alerts through **MQTT** or dashboard panels.

If you want, I can make a **complete Pi4 wiring diagram** including MLX90640, Pi Camera, RCWL-0516, and PIR sensor, optimized for your 5-day hackathon setup. This will match your previous project plans.

Do you want me to create that?

visvalization of the data of these sensors to decctect huma step by step guides fdior beginners

Absolutely! Let's create a **step-by-step beginner-friendly guide** for visualizing and fusing the data from **MLX90640 thermal sensor, Pi Camera, and RCWL-0516/PIR radar sensors** to detect humans. We'll break it into manageable steps, including **installation, reading data, processing, and visualizing on a dashboard**.

Step 1: Set Up Your Raspberry Pi 4

1. Install Raspberry Pi OS (Lite or Desktop) and update packages:

```bash
sudo apt update
sudo apt upgrade -y
```

2. Enable **I²C** (for MLX90640) and **Camera**:

```bash
```

```
sudo raspi-config
```

- Interface Options → I2C → Enable
- Interface Options → Camera → Enable

3. Install Python3 and essential libraries:

```bash
sudo apt install python3-pip python3-opencv python3-smbus i2c-tools -y
pip3 install numpy matplotlib adafruit-circuitpython-mlx90640
```

Step 2: Connect Your Sensors

- **MLX90640** → I²C (SDA/SCL, 3.3V, GND)
- **Pi Camera V2** → CSI port
- **RCWL-0516 Radar / PIR Sensor** → GPIO pins (input)
- Check I²C device:

```bash
i2cdetect -y 1
```

Should show `**0x33**` for MLX90640.

Step 3: Read Thermal Data (MLX90640)

```python
import time
import board
import busio
import adafruit_mlx90640
import numpy as np
import matplotlib.pyplot as plt

# Initialize I2C
i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)
mlx = adafruit_mlx90640.MLX90640(i2c)
mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ

frame = np.zeros((24*32,))

plt.ion()
fig, ax = plt.subplots()
im = ax.imshow(frame.reshape((24,32)), cmap='inferno', vmin=20, vmax=40)
plt.colorbar(im)
```

```
while True:
    try:
        mlx.getFrame(frame)
        im.set_data(frame.reshape((24,32)))
        plt.draw()
        plt.pause(0.01)
    except KeyboardInterrupt:
        break
```

What this does:

- Captures 24×32 thermal pixels.
- Plots live heatmap showing warm objects (like humans).

Step 4: Capture Camera Feed

python Copy code

```
import cv2

cap = cv2.VideoCapture(0)  # Pi Camera
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Camera Feed", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Optional: Integrate YOLO for human detection:

python Copy code

```
# Using pretrained YOLOv5/YOLOv8 Nano model
# pip install ultralytics
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
results = model(frame)
frame = results[0].plot()  # Draw bounding boxes
```

Step 5: Read Radar / PIR Sensor

python Copy code

```python
import RPi.GPIO as GPIO
import time

PIR_PIN = 17  # GPIO pin
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)

while True:
    if GPIO.input(PIR_PIN):
        print("Movement Detected!")
    time.sleep(0.1)
```

Step 6: Fuse Sensor Data

Basic idea:

- Thermal heatmap → detect warm areas.
- Camera YOLO → detect humans visually.
- Radar/PIR → detect movement.
- Combine with **simple logic**:

```python
if thermal_detected and radar_detected and yolo_detected:
    print("Human detected! Raise alert.")
```

Step 7: Visualize on a Dashboard

Option 1: Matplotlib / OpenCV windows

- Overlay thermal heatmap on camera feed:

```python
thermal_resized = cv2.resize(frame_thermal, (frame_cam.shape[1], frame_cam.shape[0]))
overlay = cv2.addWeighted(frame_cam, 0.6, thermal_resized, 0.4, 0)
cv2.imshow("Overlay Feed", overlay)
```

Option 2: Web-based dashboard (recommended for hackathon)

- Use **Flask + Socket.IO** to stream feeds to a browser.
- Panel can show:
  - Camera feed
  - Thermal overlay
  - Radar/PIR status
  - Alerts

Step 8: Add Alerts

- Send **Telegram / MQTT / buzzer alerts** when human detected.

```python
# Example pseudo-code
if human_detected:
    send_alert("Human detected in area!")
```

Copy code

✅ Outcome for Beginners

- You'll see **live camera feed**.
- **Thermal heatmap overlay**.
- Movement alerts from radar/PIR.
- Optional YOLO bounding boxes for humans.
- Everything displayed on your PC or web dashboard.

I can also create a **ready-to-use beginner-friendly Python template** that fuses **all three sensors** and outputs a **single dashboard with heatmap overlay + alerts**. This will be fully runnable on Pi4 with minimal setup.

Do you want me to make that template?

alerts system and overlays

Perfect! Let's build a **complete alert system with overlays** for your Raspberry Pi 4 using **MLX90640 thermal sensor, Pi Camera, and RCWL-0516/PIR radar**, step by step, for beginners. This will combine **thermal heatmaps, camera feed, human detection, and real-time alerts**.

Step 1: Hardware Connections Recap

Sensor	Pi Pin/Connection
MLX90640	SDA → GPIO2, SCL → GPIO3, VDD → 3.3V, GND → GND
Pi Camera	CSI port
RCWL-0516	OUT → GPIO17, VCC → 5V, GND → GND

Sensor	Pi Pin/Connection
Buzzer/LED	GPIO22 (buzzer), GPIO27 (LED), GND → GND

Step 2: Install Required Libraries

bash Copy code

```
sudo apt update
sudo apt install python3-pip python3-opencv python3-smbus i2c-tools -y
pip3 install adafruit-circuitpython-mlx90640 numpy matplotlib ultralytics RPi.GPIO
```

Step 3: Read Thermal Sensor and Camera Feed

python Copy code

```python
import cv2
import time
import board
import busio
import adafruit_mlx90640
import numpy as np
import RPi.GPIO as GPIO

# Radar/PIR setup
PIR_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)
BUZZER = 22
LED = 27
GPIO.setup(BUZZER, GPIO.OUT)
GPIO.setup(LED, GPIO.OUT)

# MLX90640 setup
i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)
mlx = adafruit_mlx90640.MLX90640(i2c)
mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ
thermal_frame = np.zeros((24*32,))

# Camera setup
cap = cv2.VideoCapture(0)

# YOLOv8 model
from ultralytics import YOLO
yolo_model = YOLO("yolov8n.pt")
```

```
def alert():
    GPIO.output(BUZZER, True)
    GPIO.output(LED, True)
    time.sleep(0.5)
    GPIO.output(BUZZER, False)
    GPIO.output(LED, False)

while True:
    try:
        # --- Read Thermal ---
        mlx.getFrame(thermal_frame)
        thermal_img = np.array(thermal_frame).reshape((24,32))
        thermal_img = cv2.resize(thermal_img, (640,480))
        thermal_img = cv2.applyColorMap(np.uint8((thermal_img-20)*12), cv2.COLORMAP_INFERNO)

        # --- Read Camera ---
        ret, cam_frame = cap.read()
        if not ret:
            continue

        # --- YOLO Human Detection ---
        results = yolo_model(cam_frame)[0]
        for box, cls, conf in zip(results.boxes.xyxy, results.boxes.cls, results.boxes.conf):
            if int(cls) == 0:  # Class 0 = person
                x1, y1, x2, y2 = map(int, box)
                cv2.rectangle(cam_frame, (x1,y1), (x2,y2), (0,255,0), 2)
                cv2.putText(cam_frame, f"Person {conf:.2f}", (x1, y1-5),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)

        # --- Overlay Thermal on Camera ---
        overlay = cv2.addWeighted(cam_frame, 0.6, thermal_img, 0.4, 0)

        # --- Radar/PIR Detection ---
        radar_detected = GPIO.input(PIR_PIN)
        if radar_detected:
            cv2.putText(overlay, "Radar Movement Detected!", (10,30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

        # --- Check for alerts ---
        human_detected = len(results.boxes) > 0 and radar_detected
        if human_detected:
            cv2.putText(overlay, "ALERT: HUMAN DETECTED", (10,70), cv2.FONT_HERSHEY_SIMPLEX,
1, (0,0,255), 2)
            alert()

        # --- Show overlay ---
        cv2.imshow("Human Detection Overlay", overlay)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
```

```
    except Exception as e:
        print("Error:", e)
        continue

cap.release()
cv2.destroyAllWindows()
GPIO.cleanup()
```

Step 4: How It Works

1. **Thermal Overlay** – Shows heat from MLX90640 sensor as a color heatmap on camera feed.
2. **YOLO Human Detection** – Draws green bounding boxes around humans in camera feed.
3. **Radar/PIR Detection** – Monitors movement, shown on overlay.
4. **Alerts** – If both YOLO and radar detect a human, buzzer and LED trigger with a visual alert.
5. **Overlay Visualization** – `cv2.addWeighted` merges thermal heatmap with camera feed for easy monitoring.

Step 5: Optional Enhancements

- Stream overlay to a **web dashboard** via Flask + Socket.IO.
- Log detection timestamps and coordinates.
- Fuse multiple sensors (thermal, radar, camera) for **threat scoring**.
- Add **email or Telegram alerts** for remote monitoring.

This script is **beginner-friendly** and runs entirely on Pi4 with **live overlay + alert system**, perfect for your hackathon.

I can also create a **diagram showing the overlay and alert pipeline** so it's easier to present in your SIH PPT.

Do you want me to make that diagram?

threat scoring and confidence system

Perfect! Let's design a **threat scoring and confidence system** for your Raspberry Pi 4 multi-sensor human detection setup. This will combine **thermal heatmaps, camera YOLO detection, and radar/PIR motion detection** into a single **score that indicates the likelihood of a human threat**, along with a confidence metric. I'll break it down step by step, beginner-friendly.

Step 1: Define Sensors and Confidence Weights

Each sensor contributes to the overall **threat score** based on reliability:

Sensor	Contribution	Notes
YOLO Camera	0–50 pts	Based on bounding boxes and confidence of detection (0–1)
MLX90640 Thermal	0–30 pts	Based on detected warm regions consistent with human size/shape
Radar/PIR	0–20 pts	Binary motion detection, add weight if motion detected

Total possible score = 100

- Score > 70 → High threat
- 40–70 → Medium threat
- <40 → Low threat

Step 2: Calculate Individual Sensor Confidence

a) YOLO Detection Confidence

- YOLO gives `conf` for each bounding box (0–1).
- Max YOLO confidence in the frame multiplied by 50:

```python
yolo_confidence = max([conf for conf in results.boxes.conf], default=0)
yolo_score = yolo_confidence * 50
```

b) Thermal Detection Confidence

- Count pixels in the thermal heatmap above a temperature threshold (e.g., 32°C).
- Score proportional to the percentage of hot pixels in human-size region:

```python
hot_pixels = np.sum(thermal_img > 32)
total_pixels = thermal_img.size
thermal_score = (hot_pixels / total_pixels) * 30
thermal_score = min(thermal_score, 30)
```

c) Radar/PIR Detection

- Binary motion detected → score = 20
- No motion → score = 0:

```python
radar_score = 20 if GPIO.input(PIR_PIN) else 0
```

Step 3: Compute Total Threat Score

```python
total_threat_score = yolo_score + thermal_score + radar_score

if total_threat_score > 70:
    threat_level = "HIGH"
elif total_threat_score > 40:
    threat_level = "MEDIUM"
else:
    threat_level = "LOW"

confidence_percentage = min(total_threat_score, 100)
```

Step 4: Display Overlay with Threat & Confidence

Enhance your previous overlay code to include score:

```python
cv2.putText(overlay, f"Threat: {threat_level}", (10,110), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,0,255), 2)
cv2.putText(overlay, f"Confidence: {confidence_percentage:.1f}%", (10,150),
cv2.FONT_HERSHEY_SIMPLEX, 1, (255,0,0), 2)
```

- **Red text** = high threat
- **Blue text** = confidence percentage
- Overlay now shows **visual alert + numeric score + confidence**

Step 5: Optional Enhancements

- Include **time-based scoring**: average threat score over last 5 seconds to reduce false positives.
- Scale weights based on environment (night/day, indoor/outdoor).
- Log **score history** for analytics or dashboard graph.

✅ Outcome

- **Threat Score (0–100)** = fused confidence from all sensors.
- **Threat Level** = Low, Medium, High.
- **Confidence** = numeric probability of human presence.
- Works in **real-time overlay** with thermal + camera + radar.

If you want, I can create a **ready-to-run Python script** that fuses **YOLO, MLX90640, Radar/PIR**, calculates **threat score and confidence**, and outputs **live overlay with alerts** for your Pi4 hackathon project.

Do you want me to make that complete script?

fuses scripts

Perfect! Let's create a **complete beginner-friendly Python script** for your Raspberry Pi 4 that **fuses MLX90640 thermal sensor, Pi Camera with YOLOv8, and RCWL-0516/PIR radar** to detect humans, overlay thermal + camera feed, and generate **threat scores and alerts**. This will run in real-time with a live visualization.

Complete Fused Human Detection Script with Alerts & Threat Scoring

python Copy code

```
import cv2
import time
import board
import busio
import adafruit_mlx90640
import numpy as np
import RPi.GPIO as GPIO
from ultralytics import YOLO

# ---------------------- GPIO Setup ----------------------
PIR_PIN = 17
BUZZER = 22
LED = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)
```

```
GPIO.setup(BUZZER, GPIO.OUT)
GPIO.setup(LED, GPIO.OUT)

def alert():
    GPIO.output(BUZZER, True)
    GPIO.output(LED, True)
    time.sleep(0.5)
    GPIO.output(BUZZER, False)
    GPIO.output(LED, False)

# ---------------------- MLX90640 Setup ----------------------
i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)
mlx = adafruit_mlx90640.MLX90640(i2c)
mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ
thermal_frame = np.zeros((24*32,))

# ---------------------- Camera Setup ----------------------
cap = cv2.VideoCapture(0)  # Pi Camera
yolo_model = YOLO("yolov8n.pt")  # Pretrained YOLOv8 Nano

# ---------------------- Main Loop ----------------------
try:
    while True:
        # --- Read Thermal Data ---
        try:
            mlx.getFrame(thermal_frame)
        except Exception as e:
            print("Thermal read error:", e)
            continue
        thermal_img = np.array(thermal_frame).reshape((24,32))
        thermal_img_resized = cv2.resize(thermal_img, (640,480))
        thermal_colormap = cv2.applyColorMap(np.uint8((thermal_img_resized-20)*12),
cv2.COLORMAP_INFERNO)

        # --- Read Camera Feed ---
        ret, cam_frame = cap.read()
        if not ret:
            continue

        # --- YOLO Human Detection ---
        results = yolo_model(cam_frame)[0]
        yolo_detected = len(results.boxes) > 0
        max_conf = max([conf for conf in results.boxes.conf], default=0)
        for box, cls, conf in zip(results.boxes.xyxy, results.boxes.cls, results.boxes.conf):
            if int(cls) == 0:  # Person class
                x1, y1, x2, y2 = map(int, box)
                cv2.rectangle(cam_frame, (x1,y1), (x2,y2), (0,255,0), 2)
                cv2.putText(cam_frame, f"Person {conf:.2f}", (x1, y1-5),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)
```

```
        # --- Radar/PIR Detection ---
        radar_detected = GPIO.input(PIR_PIN)
        if radar_detected:
            cv2.putText(cam_frame, "Radar Motion Detected", (10,30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

        # --- Threat Scoring ---
        # YOLO: 0-50 pts
        yolo_score = max_conf * 50
        # Thermal: 0-30 pts
        hot_pixels = np.sum(thermal_img_resized > 32)
        total_pixels = thermal_img_resized.size
        thermal_score = min((hot_pixels/total_pixels)*30, 30)
        # Radar: 0-20 pts
        radar_score = 20 if radar_detected else 0

        total_threat_score = yolo_score + thermal_score + radar_score
        confidence_percentage = min(total_threat_score, 100)

        if total_threat_score > 70:
            threat_level = "HIGH"
        elif total_threat_score > 40:
            threat_level = "MEDIUM"
        else:
            threat_level = "LOW"

        # --- Alerts ---
        if total_threat_score > 40:
            alert()

        # --- Overlay Thermal on Camera Feed ---
        overlay = cv2.addWeighted(cam_frame, 0.6, thermal_colormap, 0.4, 0)

        # --- Overlay Threat Info ---
        cv2.putText(overlay, f"Threat Level: {threat_level}", (10, 70),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)
        cv2.putText(overlay, f"Confidence: {confidence_percentage:.1f}%", (10, 110),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (255,0,0), 2)

        # --- Display Overlay ---
        cv2.imshow("Human Detection Overlay", overlay)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

except KeyboardInterrupt:
    print("Exiting program...")

finally:
```

```
    cap.release()
    cv2.destroyAllWindows()
    GPIO.cleanup()
```

Features of this Script

1. **Thermal Overlay** – MLX90640 heatmap on camera feed.
2. **YOLO Detection** – Draws bounding boxes for humans with confidence.
3. **Radar/PIR Detection** – Shows movement detection.
4. **Threat Scoring** – Fuses YOLO, thermal, and radar to compute **0–100 score**.
5. **Threat Level & Confidence** – Displayed on overlay: Low, Medium, High.
6. **Alert System** – Buzzer + LED triggers for any non-low threat.
7. **Real-time Overlay** – Combines all sensor data for live monitoring.

This is a **full hackathon-ready script** that fuses multiple sensors, provides **visual feedback**, and generates **real-time alerts with threat scoring**.

I can also create a **diagram showing the data flow** from all sensors → scoring → overlay → alerts, which would be perfect for your **SIH presentation**.

Do you want me to make that diagram?